

Unit-2
Scenario Based Assignment
CSA1717 – Artificial intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

AI-Powered Drone Delivery – Greedy vs A*

i. Greedy Best-First Search in this scenario

Takeaway: Greedy focuses only on the heuristic (straight-line distance), not actual cost.

- Greedy Best-First Search selects the node with **minimum heuristic $h(n)$** .
- In the drone scenario, it will **always chase the nearest-looking location**, even if the path is risky or blocked.
- It ignores terrain cost, weather cost, and real-time updates unless they change the heuristic.

Strengths

- Very fast → suitable for urgent situations.
- Low memory usage.
- Works well when heuristic is highly accurate.

Weaknesses

- Not optimal → may choose a dangerous or costly route.
- Easily misled by inaccurate heuristics (e.g., blocked roads).
- May get stuck in loops or dead ends.
- Cannot adapt well to dynamic changes.

ii. How A* handles the situation*

Takeaway: A* balances actual cost + heuristic → safer and more reliable.

A* uses

$$f(n) = g(n) + h(n)$$

- **g(n):** actual movement cost (terrain, weather, detours)
- **h(n):** straight-line distance heuristic

Behavior in this scenario

- A* considers both **risk** and **distance**.
- If weather increases cost, g(n) increases → A* avoids that path.
- If a road is blocked, A* recalculates f(n) and chooses a safer alternative.

Strengths

- Optimal when heuristic is admissible.
- More robust to dynamic updates.
- Safer route selection.

Weaknesses

- Slower than Greedy.
- Higher memory usage.
- Real-time recalculation may cause delay.

iii. Impact of heuristic accuracy

Greedy Best-First Search

- Highly dependent on h(n).
- If heuristic is wrong → drone chooses dangerous or blocked routes.

A*

- Less sensitive because g(n) compensates for heuristic errors.
- Still affected, but performance degrades gracefully.

iv. Effect of dynamic changes (blocked paths)

- **Greedy:**
 - Fails badly → keeps choosing nearest-looking node even if blocked.
 - Requires restarting search frequently.
- A^*
 - Updates $g(n)$ and $f(n)$ → adapts better.
 - Can replan efficiently using incremental A^* variants (D^* Lite).

v. Most suitable algorithm

Recommendation: A^* Reason:

- Balances speed + safety + optimality.
 - Handles dynamic terrain and weather better.
 - Greedy is fast but unsafe and unreliable under uncertainty.
-

Smart City Traffic Optimization – Hill Climbing & Metaheuristics

Formulation as an optimization problem

Variables: signal timings at each intersection **Domain:** possible green/red/yellow durations

Objective function: minimize

- total waiting time
- fuel consumption
- congestion

Why traditional search is inefficient

- Search space is enormous (hundreds of intersections $\times$ timings).
- Dynamic traffic → state changes continuously.
- No fixed goal state → continuous optimization needed.

i. Hill Climbing in this scenario

- Starts with a current signal configuration.
- Makes small adjustments (neighbors).
- Moves to the configuration with best improvement.

Limitations

- Gets stuck in local maxima.
- Cannot explore radically different configurations.
- No backtracking → poor for dynamic traffic.

ii. Issues: Local maxima, plateaus, ridges

- **Local maxima:**
 - A timing pattern that is better than neighbors but not globally optimal.
 - Example: one junction optimized but city-wide congestion remains.
- **Plateaus:**
 - Many configurations give same performance → no direction to move.
 - Example: night-time low traffic.
- **Ridges:**
 - Requires long sequence of small changes → hill climbing fails.
 - Example: coordinated timing across multiple intersections.

iii. How Simulated Annealing / Genetic Algorithms help

Simulated Annealing

- Allows “bad moves” early → escapes local maxima.
- Gradually reduces randomness → converges to good solution.

Genetic Algorithms

- Population-based search.

- Crossover mixes good solutions.
- Mutation introduces randomness → avoids plateaus.
- Highly scalable for large cities.

iv. Best approach

Recommendation: Genetic Algorithms Reason:

- Scalable to hundreds of intersections.
 - Adapts well to dynamic traffic.
 - Avoids local maxima using population diversity.
-

Mars Rover Exploration – Online Search & Partial Observability

i. Why online search is required

- Rover does not have a complete map.
- Must act **before** knowing full environment.
- Offline search requires full state space → impossible on Mars.

Online search = **interleave planning + execution + learning.**

ii. Challenges of partial observability & dynamic environment

- Sensors have limited range → rover sees only nearby terrain.
- Hazards appear unexpectedly.
- Rover must update beliefs continuously.
- Decisions must be safe even with incomplete information.

iii. How rover builds and updates internal model

- Maintains a **belief map** of explored areas.
- Uses sensor data to mark safe zones, hazards, unknown areas.

- Updates cost estimates as new terrain is discovered.
- Uses algorithms like LRTA* or D* Lite for real-time replanning.

iv. Compare online search with uninformed & informed search

- **Uninformed search:**
 - Blind, inefficient, cannot handle unknown terrain.
- **Informed search:**
 - Uses heuristics but still needs full map.
- **Online search:**
 - Learns while exploring.
 - Replans dynamically.
 - Best suited for unknown environments.

v. Most suitable approach

Recommendation: LRTA\ or D\ Lite (online heuristic search)** **Reason:**

- Handles uncertainty.
 - Continuously updates map.
 - Safe navigation with limited sensing.
 - Proven effective in real robotic exploration.
-

Exam Timetable Scheduling – CSP

Formulation as CSP

Variables:

- Exam slots for each course
- Hall assignments
- Faculty availability

- Special accommodation slots

Domains:

- Possible time slots
- Available halls
- Valid faculty schedules

Constraints:

- No student has overlapping exams.
- Hall capacity limits.
- Precedence constraints (subject A before B).
- Faculty availability.
- Special accommodations.

i. Backtracking search

- Assign a variable (course $\rightarrow$ time slot).
- Check constraints.
- If conflict $\rightarrow$ backtrack.
- Systematically explores assignments.

ii. Constraint propagation (forward checking)

- After assigning a variable, eliminate invalid values from future domains.
- Prevents exploring impossible paths.
- Reduces search drastically.

iii. Real-world challenges

- Large number of students and courses.
- Many overlapping constraints.
- Dynamic changes (faculty leave, hall unavailability).

iv. Best strategy

Recommendation: Backtracking + Forward Checking + MRV heuristic Reason:

- Scalable.
 - Efficient pruning.
 - MRV chooses the most constrained variable first → faster solution.
-

Real-Time Strategy Game AI – Minimax & Alpha-Beta

i. How Minimax models the problem

- Game represented as a decision tree.
- Max player (AI) tries to maximize score.
- Min player (human) tries to minimize AI score.
- Minimax chooses the best move assuming optimal opponent.

ii. Computational challenges

- Huge branching factor (many units, actions).
- Deep game tree → exponential complexity.
- Real-time constraints → cannot explore full tree.

iii. How Alpha-Beta pruning improves efficiency

- Eliminates branches that cannot influence final decision.
- Uses α (best for max) and β (best for min).
- If a branch is worse than current α or β → prune.
- Reduces nodes explored from $O(b^d)$ to nearly $O(b^{d/2})$.

iv. Evaluation function design

Should consider:

- Material advantage (units, resources).

- Map control (territory).
- Unit health and strength.
- Strategic positions (high ground, safe zones).
- Potential future threats.

Justification: These factors directly influence winning probability in real-time strategy games.

v. Best strategy for balancing optimality & real-time performance

Recommendation: Iterative Deepening + Alpha-Beta + Good Evaluation Function

Reason:

- Ensures best possible move within time limit.
- Alpha-Beta reduces computation.
- Evaluation function gives quick approximate value.
- Works well under strict real-time constraints.